



HIGHER-LEVEL WORKLOADS

Book 06 of the series — completing the workload zoo: DaemonSets (a Pod on every node), Jobs & CronJobs (run-to-completion and schedules), StatefulSets in full (ordered lifecycle + partition canaries), and the autoscaling trio — HPA, VPA, Cluster Autoscaler.



DaemonSet

one Pod per node



Job

run to done



CronJob

on a schedule



StatefulSet

full lifecycle



HPA

more replicas



VPA

bigger replicas



Cluster Autoscaler

more nodes



Canaries

partition updates

PART I · DAEMONSETS

- ☐ **CH 01** DaemonSet — One Pod Per Node p.3
- ☐ **CH 02** DaemonSet — Updates & Node Lifecycle ... p.4

PART II · JOBS & CRONJOBS

- ☐ **CH 03** Jobs — Run to Completion p.5
- ☐ **CH 04** CronJobs — Schedules & Concurrency p.6

PART III · STATEFULSETS IN FULL

- ☐ **CH 05** Ordered Lifecycle & Stable Identity p.7
- ☐ **CH 06** Update Strategies — Partition Canaries p.8

PART IV · AUTOSCALING

- ☐ **CH 07** HPA — Horizontal Pod Autoscaler p.9
- ☐ **CH 08** VPA — Vertical Autoscaler (Overview) p.10
- ☐ **CH 09** Cluster Autoscaler — Nodes on Demand .. p.11
- ☐ **CH 10** The Trio Together — When to Use Which .. p.12

PART V · LABS

- ☐ **LAB 1** DaemonSet on Every Node p.13
- ☐ **LAB 2** Jobs & a Manual CronJob Trigger p.14
- ☐ **LAB 3** HPA — Watch It Scale Under Load p.15
- ☐ **LAB 4** StatefulSet Partition Canary p.16

PART VI · PROVE IT

- ☐ **QUIZ** Self-Check Quiz p.17
- ☐ **KEY** Answer Key + Scoring p.18
- ☐ **GLOSS** Glossary p.19
- ☐ **CHEAT** One-Page Cheat Sheet p.20
- ☐ **FINAL** Where to Next → Book 07 p.21



Assumes Books 01–05. You've already *met* everything here in passing: daemon-like agents skipped your drains (Book 04 · CH 10), Promtail/node-exporter ride DaemonSets (your observability stack!), and Book 05 built the StatefulSet storage half. This book finishes each story — then makes workloads **run themselves**.



A Deployment says "run *N* replicas, I don't care where." A DaemonSet says "run exactly *ONE* Pod on *EVERY* node — new node joins, Pod spawns; node leaves, Pod vanishes." Infrastructure agents live here.

DEPLOYMENT VS DAEMONSET — THE PLACEMENT QUESTION

Deployment "3 total, anywhere" ⇔

DaemonSet "1 per node, everywhere"



node1



node2



node3



DaemonSet

every node, exactly one

```
apiVersion: apps/v1
kind: DaemonSet
metadata: { name: log-agent }
spec:
  selector: { matchLabels: { app: log-agent } }
  template:
    # no `replicas:` field — the NODE LIST decides
    metadata: { labels: { app: log-agent } }
    spec:
      containers:
        - name: agent
          image: fluent/fluent-bit:2.2
          volumeMounts:
            - { name: logs, mountPath: /var/log/pods, readOnly: true }
      volumes:
        - name: logs
          hostPath: { path: /var/log/pods } # node-local files (Book 05 · CH 02!)
```

The four classic tenants

Why per-node is mandatory

Log collectors — Fluent Bit, Promtail	container logs are files on each node's disk — only a local Pod can tail them (Book 01 · CH 03c logic)
Metric agents — node-exporter, Datadog agent	host metrics only exist at the node
Networking — CNI plugins, kube-proxy	every node needs routing rules
Storage — CSI node plugins, Longhorn, GlusterFS	every node needs the mount machinery (Book 05 · CH 11!)



You've been commanding DaemonSets for two books: `drain --ignore-daemonsets` (Book 04 · CH 10) — their Pods are node-bound, so drain skips them. And your Loki stack's **Promtail runs as a DaemonSet** for exactly the `/var/log/pods` reason above.



One sentence: a DaemonSet is "*infrastructure agents as YAML*" — one Pod per node, born on join, gone on leave, no replicas field because **the node list IS the replica count**.

🔊 Updates, taints, and node churn — [CH 02: DaemonSet Lifecycle](#)

↑ back to top

**NEW Node churn — the automatic part**

node4 joins → kubelet registers → DaemonSet controller notices → **Pod spawns** within seconds
node4 dies → node object removed (or heartbeat lost) → **Pod garbage-collected**
node tainted? → DaemonSets auto-tolerate the **infrastructure** taints:
 not-ready · unreachable · disk-pressure · memory-pressure ✓ auto
 control-plane:NoSchedule ✗ NOT auto – add the toleration yourself

```
# run your DS on control-plane nodes too (Book 04 · CH 05 taints, in action):
tolerations:
- { key: node-role.kubernetes.io/control-plane, operator: Exists, effect: NoSchedule }
```

🎬 updateStrategy — rolling your agents

Strategy	Behavior	Notes
RollingUpdate default	node by node, controlled by maxUnavailable	no maxSurge for DaemonSets — agents replace in place, brief per-node gap is fine for infra
OnDelete	updated only when <i>you</i> delete each Pod	manual control for scary infrastructure (CNI!)

```
$ kubectl set image ds/log-agent agent=fluent/fluent-bit:2.3
daemonset.apps/log-agent image updated
$ kubectl rollout status ds/log-agent
daemon set "log-agent" successfully rolled out
```

nodeSelector / affinity respected: add `nodeSelector: {gpu: "true"}` → the DaemonSet only covers GPU nodes — *"one Pod per matching node"* is the honest definition

revisionHistoryLimit keeps old controller revisions — **rollout undo** works on DaemonSets too



Handle the control-plane exception: forgetting the toleration above means your "cluster-wide" agent silently runs on *workers only* — and on single-node minikube (one control-plane node), it runs **nowhere**. Lab 1 rides this trap on purpose.



One sentence: DaemonSets ride the node lifecycle automatically — *join* → *spawn*, *leave* → *collect* — auto-tolerate infrastructure sickness but **not the control-plane taint**, and roll out node-by-node like Deployments minus the surge.

🔊 Workloads that END — [CH 03: Jobs](#)

↑ back to top



Everything so far runs *forever* — servers don't exit. A **Job** runs Pods to **completion**: migrate a schema, render a video, process yesterday's files. Success = exit 0. Then it stops.

```
apiVersion: batch/v1
kind: Job
metadata: { name: nightly-etl }
spec:
  completions: 5           # run 5 SUCCESSFUL pods total
  parallelism: 2           # ...but only 2 at a time
  backoffLimit: 4          # allow 4 failures before the JOB fails
  activeDeadlineSeconds: 3600 # hard wall-clock kill-switch
  ttlSecondsAfterFinished: 86400 # auto-cleanup 24h after done
  template:
    spec:
      restartPolicy: Never    # OnFailure or Never — NEVER Always!
      containers:
      - name: etl
        image: myorg/etl:3.1
```

Field	Question it answers
completions / parallelism	how many successes total / how many concurrent — omitted → 1 and 1
backoffLimit	how many failed Pods before declaring the whole Job failed
activeDeadlineSeconds	when to give up regardless of retries
ttlSecondsAfterFinished	when the Job and its Pods clean themselves up
completionMode: Indexed	each Pod gets index <code>JOB_COMPLETION_INDEX</code> — sharding a work queue

```
$ kubectl get job,pods -w
NAME                COMPLETIONS  DURATION  AGE
nightly-etl         2/5          45s       45s
NAME                STATUS      RESTARTS
nightly-etl-abc     Running     0
nightly-etl-def     Running     0
nightly-etl-ghi     Completed   0 # exit 0 — counts toward 5
```



restartPolicy must be Never or OnFailure — a Job template with `Always` is rejected. Think about it: a *completed* workload can't have an "always restart" policy. The Book 01 · CH 15 standalone-Pod problem also applies: a Job whose **node dies** gets its Pod recreated by *the Job controller* — a plain Pod wouldn't be.



One sentence: a Job is a batch workload with an exit condition — it counts **successes** (`completions`), bounds **failure** (`backoffLimit`), caps **time** (`activeDeadlineSeconds`), and cleans up after itself (`ttlSecondsAfterFinished`).

▶▶ The same thing, on a clock — [CH 04: CronJobs](#)

↑ back to top



A CronJob creates a **Job on a schedule** — cron syntax you already know, wrapped around everything CH 03 taught.

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: db-backup }
spec:
  schedule: "30 2 * * *"           # 02:30 daily (min hour dom month dow)
  timeZone: "Europe/Berlin"       # stable zone – don't trust node clocks
  concurrencyPolicy: Forbid        # ★ the policy that saves databases
  startingDeadlineSeconds: 300     # missed > 5 min ago → skip, don't stampede
  suspend: false                  # true = pause the schedule, keep the object
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:                    # ← everything from CH 03 lives in here
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: dump
              image: postgres:16
              command: [pg_dump, -f, /backup/dump.sql, $DATABASE_URL]
```

concurrencyPolicy	If the previous run is still going...	Use for
Allow default	start another anyway — runs overlap	idempotent jobs (cache warmers)
Forbid	skip this tick	backups, billing — two overlapping runs = disaster
Replace	kill the old run, start new	"latest state wins" jobs



The overlap trap: a backup that takes 25 min on a 15-min schedule, with `Allow`, silently runs *two backups against the same database*. `Forbid` is the professional default for anything touching shared state — and `startingDeadlineSeconds` prevents a recovered controller from firing 47 missed ticks at once.

```
# fire a schedule right now – the tester's cheat code:
$ kubectl create job backup-test --from=cronjob/db-backup
job.batch/backup-test created
$ kubectl get jobs # backup-test 0/1 → 1/1 – full CH 03 machinery
```



One sentence: a CronJob = **cron schedule** + **Job template** + a **concurrency policy** — and the entire craft is choosing `Allow/Forbid/Replace` correctly, because overlapping batch runs are where data gets hurt.

▶ Part III: the cattle-and-names workload, in full — [CH 05: StatefulSets](#)

↑ back to top



StatefulSets — Ordered Lifecycle & Identity

Book 05 gave StatefulSets their storage (`volumeClaimTemplates`, per-replica claims). This page completes the other half: **ordered lifecycle** and **stable identity** — the two things a replica set of a database can't live without.

```
StatefulSet db · replicas: 3
```

```
SCALE UP (ordered):
```

```
db-0 created → waits for Running & Ready → db-1 → ... → db-2
```

```
SCALE DOWN (reverse):
```

```
db-2 deleted first → then db-1 → db-0 last (primary dies last!)
```

```
DELETION of the STS: same reverse order — and claims SURVIVE (Book 05 · CH 10)
```

STABLE NETWORK IDENTITY

```
# via the HEADLESS svc (Book 05 · CH 09):
db-0.db.shop.svc.cluster.local
db-1.db.shop.svc.cluster.local
db-2.db.shop.svc.cluster.local
# names survive rescheduling —
# "follow db-0" is a TRUE forever
```

PODMANAGEMENTPOLICY

```
OrderedReady (default)
strict sequence, wait-for-Ready —
correct for most databases
Parallel
spawn/kill all at once — for
peers that self-coordinate
(e.g. some distributed DBs)
```

- Why ordered matters:** a primary must *Ready* before followers attach to it — Parallel would race followers against a half-booted primary
- Re-created in place:** kill db-1 → the **same name** returns, reattaches `data-db-1` (Book 05 · Lab 3 proved the data comes back) — the identity is the contract
- Cascade control:** `kubectl delete sts db -- cascade=orphan` leaves the Pods running — hand-migrate them, then adopt



One sentence: StatefulSets = **ordered birth/reverse-order death + wait-for-Ready between steps + names and DNS that never change** — storage (Book 05) plus sequence plus identity, the full definition of "stateful."

Updating these delicate creatures safely — [CH 06: Partition Canaries](#)

↑ back to top



Rolling a database's image is scarier than rolling nginx. StatefulSets answer with **partitioned rollouts**: update a controlled slice first, verify, then let it finish.

```
spec:
  updateStrategy:
    type: RollingUpdate          # or: OnDelete (manual, like DaemonSets)
    rollingUpdate:
      partition: 2              # ONLY pods with ordinal ≥ 2 update
```

PARTITION: 2 ON A 3-REPLICA STATEFULSET — AFTER SET IMAGE

db-0
postgres:16 · old

db-1
postgres:16 · old

db-2
postgres:17 · NEW 🌸

the canary flies solo — verify it, then lower the partition to release the rest

```
# the canary release, step by step:
$ kubectl patch sts db --type=merge -p \
  '{"spec":{"updateStrategy":{"rollingUpdate":{"partition":2}}}}'
$ kubectl set image sts/db db=postgres:17
$ kubectl get pod db-2 -o jsonpath='{.spec.containers[0].image}'
postgres:17 # db-0/db-1 untouched ✓
$ # ...run your checks against db-2... then release:
$ kubectl patch sts db --type=merge -p \
  '{"spec":{"updateStrategy":{"rollingUpdate":{"partition":0}}}}'
$ kubectl get pods -w
db-2 1/1 Running (already new)
db-1 0/1 Terminating → 1/1 on :17 # highest ordinal first!
db-0 0/1 Terminating → 1/1 on :17
```

- ▼ **Roll direction is highest-ordinal-first** — replicas (followers) update before the primary (db-0). Followers run the new version against the old primary — the safe direction for replication compat

🌸 `partition = replicas` → **nothing updates** (freeze). Lower gradually: 2 → 1 → 0

👉 **OnDelete** — full manual: delete each Pod yourself to pull the new template (CNI/CSI-adjacent caution)



Compare to Deployments: Deployments have `maxUnavailable/maxSurge` (Book 01 · CH 20) — availability control. StatefulSets have `partition` — **blast-radius control**. One Pod at a time, ordered, canary-able: exactly what replicas of a database deserve.



One sentence: `partition` turns a StatefulSet rollout into a **graduated canary from the top down** — verify one replica on the new version before you let the primary anywhere near it.

▶▶ Part IV: workloads that resize themselves — [CH 07: HPA](#)

↑ back to top



The HPA watches a metric and edits `spec.replicas` for you — **more copies under load, fewer when quiet**. One controller, one loop (Book 01's reconciliation, aimed at sizing).

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: web-hpa }
spec:
  scaleTargetRef:
    { apiVersion: apps/v1, kind: Deployment, name: web }
  minReplicas: 2           # floor — never below (HA!)
  maxReplicas: 10          # ceiling — your wallet's guard rail
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 60 }
      # "keep average CPU at 60% OF EACH POD'S REQUEST"
```



HPA needs `requests` **set** (Book 02 · CH 14). Utilization = *actual usage* ÷ *request* — no request means no denominator, and the HPA sits **unknown**. It also needs the **metrics-server** installed (`minikube addons enable metrics-server`) — the component that aggregates real usage.



The math (it's one formula)

`desiredReplicas = ceil[ currentReplicas × (currentMetric ÷ targetMetric) ]`

4 pods at 90% CPU, target 60%: `ceil(4 × 90/60) = 6` pods

2 pods at 15%, target 60%: `ceil(2 × 15/60) = 1` pod

...but within a ±10% tolerance band — no flapping on noise.

...always clamped to `[minReplicas, maxReplicas]`.



Asymmetric by design:

check every ~15s;

scale-up is fast

, scale-down waits

(default stabilization window 5 min) — traffic spikes shouldn't mass-terminate Pods they may need again



Metric sources:

`Resource`

(cpu/memory per-pod) ·

`Pods`

(custom per-pod: requests/sec) ·

`External`

(queue depth, cloud LB RPS)



Scale targets: Deployment, ReplicaSet, StatefulSet — **not** DaemonSets (node-bound by definition)



One sentence: the HPA is a **replica thermostat** — it divides observed load by your `requests`-based target and scales within your min/max rails, fast up, slow down. Lab 3 makes the needle move in front of you.



Scaling a different axis — [CH 08: VPA](#)

↑ back to top



The HPA answers "wrong *NUMBER* of copies." The VPA answers "wrong *SIZE* of copy" — it watches actual usage and adjusts each Pod's **requests/limits** (Book 02 · CH 14).

HORIZONTAL (HPA)

   → load →      

same size, more of them

VERTICAL (VPA)

 (200m) → load →  (500m)

same count, bigger each

the VPA object — installed separately (CRDs + recommender component):

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: { name: web-vpa }
spec:
  targetRef: { apiVersion: apps/v1, kind: Deployment, name: web }
  updatePolicy:
    updateMode: Auto    # Off | Initial | Recreate (Auto)
```

Mode	Behavior
Off	recommendations only — the right starting point: read <code>kubectl describe vpa</code> , tune your YAML yourself
Initial	apply recommended requests only to newly created Pods — no disruption
Auto / Recreate	evicts Pods to apply new sizes (Book 01's eviction, by an autoscaler this time)



The classic conflict: HPA on **cpu** + VPA on **cpu** = a feedback fight (VPA raises requests → utilization % falls → HPA scales down...). Allowed combo: **HPA on custom/external metrics** + **VPA on cpu/memory** — different signals, no loop. This is also why "just install everything" autoscaling is a mistake.



One sentence: the VPA is a **right-sizing engine** — same knob as Book 02's requests/limits, turned automatically; start in **Off** mode and treat its recommendations as the end of guesswork, not as an autopilot.

▶ And when the NODES run out — [CH 09: Cluster Autoscaler](#)

↑ back to top



HPA and VPA resize *within* the cluster's capacity. The **Cluster Autoscaler (CA)** resizes the capacity itself — adding and removing **nodes**, driven by two signals:

SCALE UP trigger: Pods stuck Pending — Book 04's funnel found NO feasible node —
(requests don't fit / taints / affinity)

CA asks the cloud provider (node group / ASG / MIG) for one more node →
node joins → those Pending pods schedule ✓

SCALE DOWN trigger: a node is underutilized ~10+ min AND every Pod on it
can be safely rescheduled onto other nodes → drain-style eviction →
node removed → bill shrinks ✓

Pods CA will NOT evict (block scale-down)	Why
Pods under a restrictive PDB (Book 04 · CH 10)	voluntary-disruption protection is honored
Pods with local storage (emptyDir/hostPath) — unless <code>cluster-autoscaler.kubernetes.io/safe-to-evict: "true"</code>	eviction = data loss (Book 05 · CH 02!)
High-priority / system Pods (Book 04 · CH 09)	pecking order respected
Pods the controller wouldn't recreate elsewhere (anti-affinity pinned, standalone)	they'd strand



Configuration is per cloud, per node group: you annotate/label node groups with `min / max` sizes and deploy the CA with your provider's flags — it's cloud-native by design. **On minikube it doesn't function** (no expandable node group behind it) — this is a production-environment component, which is why this chapter is an overview.



One sentence: the CA is **node-count automation** — Pending Pods are its "buy more" signal, idle nodes its "sell some" signal, and PDBs/local-storage/priority are the fences it will never jump (Book 04's safety rules, enforced by an accountant).

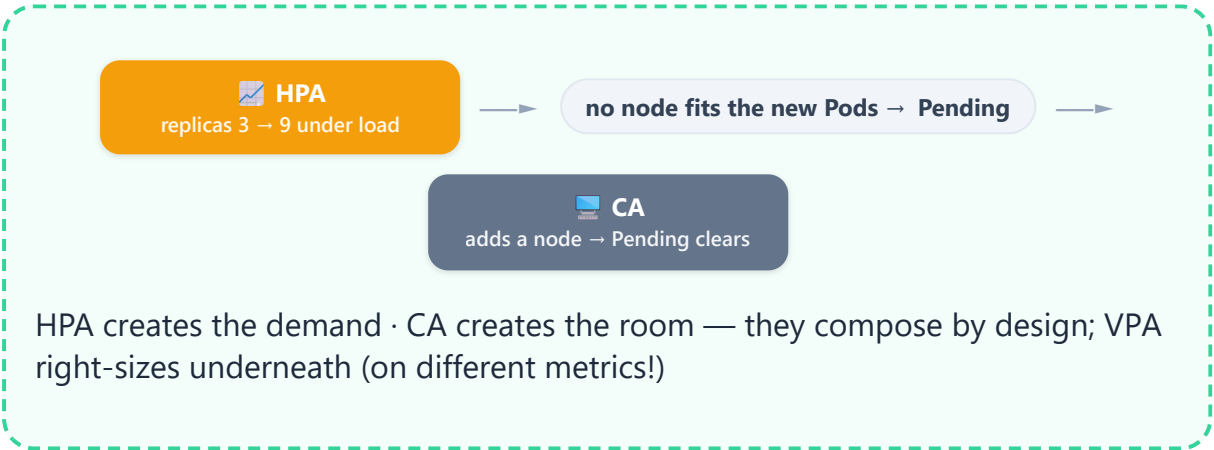
▶▶ Three scalers, one stack — [CH 10: The Trio Together](#)

↑ back to top

Three autoscalers, three **different questions**. Confusing them is how clusters get expensive or fall over.

	 HPA	 VPA	 Cluster Autoscaler
Question	how many copies?	how big is a copy?	how many nodes?
Adjusts	<code>spec.replicas</code>	requests/limits (via eviction)	node-group size (cloud)
Signal	cpu/mem utilization, custom metrics	observed usage history	Pending Pods / idle nodes
Layer	Pods	Pods	Nodes
Latency	~15 s checks	recommend + evict (minutes)	minutes (cloud API)

How they stack (the beautiful part)



Traffic-shaped load?
→ HPA (resource or custom metric)

Steady load, badly sized pods?
→ VPA in `Off` → fix YAML, maybe `Initial`

Batch bursts, cost control?
→ CA with sane group min/max

 **The two classic misconfigurations:** (1) HPA + VPA on the **same resource metric** — feedback war (CH 08). (2) HPA `minReplicas` higher than the cluster can host, with no CA — **permanent Pending** (Book 04 · CH 01's funnel, now self-inflicted by an autoscaler).

 **Remember the ladder:** *HPA counts copies · VPA sizes copies · CA buys the building floor they stand on* — pick per question, never stack two scalers on the same signal, and the whole stack runs on Book 02's honest `requests`.

 Prove all of it with your hands — [Part V: Labs](#)

↑ back to top

**STEP 1 · Deploy an agent — and feel the control-plane taint**

```
$ kubectl create ds agent --image=busybox:1.36 \
-- sh -c 'sleep 9999'
$ kubectl get ds agent
NAME    DESIRED  CURRENT  READY
agent   0         0         0 # ZERO?! single control-plane node,
# no toleration → runs NOWHERE (CH 02's trap!)
```

STEP 2 · Add the toleration — it appears

```
$ kubectl patch ds agent --type=merge -p \
'{"spec":{"template":{"spec":{"tolerations":[{"key":
"node-role.kubernetes.io/control-plane","operator":"Exists",
"effect":"NoSchedule"}]}}}}'
$ kubectl get ds agent
NAME    DESIRED  CURRENT  READY  NODE SELECTOR
agent   1         1         1      <none> # one per node — minikube has one node
```

∞ LIMIT TO A SUBSET

```
$ kubectl patch ds agent --type=merge -p \
'{"spec":{"template":{"spec":{"nodeSelector":
{"role":"gpu"}}}}}'
$ kubectl get ds agent
DESIRED
0 # no node has role=gpu —
# "1 per MATCHING node" (CH 02)
```

🔍 ROLL IT

```
$ kubectl set image ds/agent \
busybox=busybox:1.36-musl
$ kubectl rollout status ds/agent
successfully rolled out
# same machinery as deployments,
# node by node (CH 02)
```

Proved: DaemonSets count nodes, auto-tolerate nothing for the control plane, honor nodeSelector, and roll like Deployments. **Teardown:** `kubectl delete ds agent`.

[↑ back to top](#)

**STEP 1 · A counted Job — 4 successes, 2 at a time**

```
$ kubectl create job counted --image=busybox:1.36 \
-- sh -c 'echo working; sleep 5'
$ kubectl patch job counted --type=merge -p \
'{"spec":{"completions":4,"parallelism":2}}'
$ kubectl get job counted -w
NAME      COMPLETIONS
counted   4/4 # successes counted – pods came and went
$ kubectl get pods | grep counted
(pods remain as history – Completed; that's what TTL is for)
```

*** FAILURE PATH**

```
$ kubectl create job broken \
--image=busybox:1.36 \
-- sh -c 'exit 1'
$ kubectl get job broken -w
COMPLETIONS DURATION AGE
0/1          30s
$ kubectl describe job broken | tail -1
Job has reached the
specified backoff limit
(default 6)
# 6 failed pods → JOB failed.
# backoffLimit in action
```

CRONJOB, FIRED BY HAND

```
$ kubectl create cronjob ticker \
--image=busybox:1.36 \
--schedule="*/1 * * * *" \
-- sh -c 'date'
$ kubectl create job tick-now \
--from=cronjob/ticker
$ kubectl get jobs
tick-now 1/1 Complete
# schedule skipped –
# same machinery, on demand
```

Proved: completions/parallelism counting, backoffLimit killing a hopeless Job, and the `--from=cronjob/` test trigger.

Teardown: `kubectl delete job counted broken tick-now; kubectl delete cronjob ticker`.

[↑ back to top](#)



STEP 1 · Metrics-server + a burner deployment

```
$ minikube addons enable metrics-server
$ kubectl top nodes # wait ~1 min – proves the pipeline works
$ kubectl create deploy burner --image=polinux/stress \
  -- stress --cpu 1 --timeout 900s
$ kubectl set resources deploy burner --requests=cpu=100m
# request 100m, burns 1000m → 1000% utilization
```

STEP 2 · Arm the HPA — target 50%

```
$ kubectl autoscale deploy burner --min=1 --max=6 --cpu-percent=50
horizontalpodautoscaler.autoscaling/burner autoscaled
$ kubectl get hpa burner -w
```

NAME	REFERENCE	TARGETS	REPLICAS
burner	Deployment/burner	1030%/50%	1 # usage vs REQUEST (CH 07!)
burner	Deployment/burner	1030%/50%	6 # ceil(1 × 1030/50) = 21 → clamped to max 6

WATCH THE MATH

```
# each pod still burns 1000%
# of its 100m request →
# with 6 pods, avg ≈ 1000%
# desired = ceil(6 × 20) = 120
# → stays pinned at max 6.
# THE CEILING HOLDS (wallet rail)
```

COOL-DOWN (slow down!)

```
$ kubectl delete deploy burner # or wait out the 90
# on real cool-down: scale-DOWN
# waits ~5 min (stabilization
# window) – CH 07's asymmetry,
# protecting against spikes
```

Proved: utilization is measured against `requests` ; the formula overshoots and clamps at `maxReplicas` ; metrics-server feeds it all. **Teardown:** `kubectl delete hpa burner; kubectl delete deploy burner` .

[↑ back to top](#)

**STEP 1 · A 3-replica StatefulSet (Book 05 · Lab 3 pattern, condensed)**

```
$ kubectl create deploy web --image=nginx:1.25 --replicas=3 2>/dev/null
$ kubectl delete deploy web 2>/dev/null
$ kubectl apply -f - <<EOF
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: web }
spec:
  serviceName: web
  replicas: 3
  selector: { matchLabels: { app: webs } }
  template:
    metadata: { labels: { app: webs } }
    spec:
      containers:
      - { name: nginx, image: nginx:1.25 }
EOF
$ kubectl get sts web # READY 3/3 — ordered birth (CH 05)
```

STEP 2 · Set the partition BEFORE the new image

```
$ kubectl patch sts web --type=merge -p \
'{"spec":{"updateStrategy":{"rollingUpdate":{"partition":2}}}}'
$ kubectl set image sts/web nginx=nginx:1.26
$ kubectl get pods -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image
NAME      IMAGE
web-0     nginx:1.25 # frozen — ordinal < partition
web-1     nginx:1.25 # frozen
web-2     nginx:1.26 # THE CANARY 🐦
```

STEP 3 · Release — highest ordinal rolls first

```
$ kubectl patch sts web --type=merge -p \
'{"spec":{"updateStrategy":{"rollingUpdate":{"partition":0}}}}'
$ kubectl get pods -w
# web-1 replaces first, THEN web-0 — replicas before primary (CH 06)
$ kubectl get pods -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image
web-0     nginx:1.26 web-1 nginx:1.26 web-2 nginx:1.26 ✓
```

Proved: partition gates the rollout by ordinal; release = lowering the partition; direction is top-down. **Teardown:** `kubectl delete sts web` (no PVCs here — none were created).

[↑ back to top](#)



Q1. A DaemonSet's Pod count is determined by...

- A.** spec.replicas **B.** the number of matching nodes **C.** the HPA **D.** the highest ordinal ever created

Q2. Valid restartPolicy for a Job template?

- A.** Always **B.** Always or Never **C.** Never or OnFailure **D.** none — Jobs don't restart

Q3. CronJob with concurrencyPolicy: Replace, previous run still active, new tick fires:

- A.** old run is killed, new run starts **B.** new tick is skipped **C.** both run overlapped **D.** CronJob suspends itself

Q4. StatefulSet RollingUpdate with partition: 2, 3 replicas, image changed:

- A.** all pods update **B.** only db-0 updates **C.** nothing updates **D.** only db-2 updates

Q5. When a StatefulSet rollout completes (partition 0), the update order is...

- A.** db-0 → db-1 → db-2 **B.** db-2 → db-1 → db-0 (highest ordinal first) **C.** random **D.** all simultaneously

Q6. An HPA with type: Utilization shows unknown. Most likely cause?

- A.** Pods lack resource requests (or metrics-server missing) **B.** maxReplicas too low **C.** wrong apiVersion **D.** the Deployment is behind a Service

Q7. DaemonSet Pods automatically tolerate which taints?

- A.** all taints including control-plane **B.** none — explicit only **C.** infrastructure conditions (not-ready, disk-pressure...) but NOT control-plane **D.** only PreferNoSchedule

Q8. The Cluster Autoscaler adds nodes primarily when...

- A.** CPU per node exceeds 80% **B.** Pods are Pending because no node can fit them **C.** the HPA hits maxReplicas **D.** a node reports NotReady

Q9. VPA in Auto mode applies new requests by...

- A.** editing the Deployment template in place, no restarts **B.** only new pods **C.** asking the HPA **D.** evicting Pods so replacements start with the recommended size

Q10. HPA + Cluster Autoscaler together:

- A.** conflict — choose one **B.** both manage replicas **C.** compose — HPA creates more Pods, CA adds nodes to fit them **D.** only work on StatefulSets

[↑ back to top](#)



Q	Ans	Why — one line
1	B	No replicas field — the (matching) node list decides — CH 01
2	C	Always is illegal for run-to-completion work — CH 03
3	A	Replace = kill old, start new; Forbid skips; Allow overlaps — CH 04
4	D	Partition gates by ordinal: only pods $\geq$ partition update — CH 06
5	B	Highest-ordinal-first — followers before the primary — CH 06
6	A	Utilization = usage $\div$ request; missing pieces $\rightarrow$ unknown — CH 07
7	C	Infrastructure taints auto; control-plane is opt-in — CH 02, Lab 1
8	B	Pending-for-no-room is the buy signal — CH 09
9	D	requests can't be edited live — evict & recreate — CH 08
10	C	Different layers: replicas (HPA) vs nodes (CA) — CH 10

**9–10**

Ready for Book 07

**6–8**Re-read missed chapters' 
boxes**<6**

Re-run Labs 1–4, then retake



Missed Q4/Q5? Partition + roll direction is *the* StatefulSet update mental model — canary from the top down.
 Missed Q6? HPA without `requests` is the #1 "why won't it scale" ticket in the wild.

[↑ back to top](#)



DaemonSet — one Pod per matching node; node joins spawn, node leaves collect; agent workloads (logs, metrics, CNI, CSI node plugins).

updateStrategy (DS) — RollingUpdate (maxUnavailable; no maxSurge) or OnDelete.

Job — run-to-completion workload: completions, parallelism, backoffLimit, activeDeadlineSeconds, ttlSecondsAfterFinished, completionMode Indexed.

CronJob — schedule + Job template; concurrencyPolicy (Allow/Forbid/Replace), startingDeadlineSeconds, suspend, timeZone.

StatefulSet — ordered lifecycle, stable names/DNS, per-replica storage (Book 05); podManagementPolicy OrderedReady/Parallel.

partition — StatefulSet canary gate: only ordinals $\geq$ partition update; roll direction highest-first.

OnDelete — update strategy: template applies only when you delete each Pod manually.

HorizontalPodAutoscaler — scales replicas from metrics; v2 API; needs metrics-server + Pod requests; min/max rails; fast up, slow down (stabilization window).

Utilization — actual usage $\div$ resource request; the default HPA metric basis.

VerticalPodAutoscaler — recommends/applies requests+limits; modes Off / Initial / Auto(Recreate); conflicts with HPA on the same metric.

Cluster Autoscaler — node-count automation: adds nodes for Pending Pods, removes underutilized nodes ~ 10 min; respects PDBs, local storage (safe-to-evict annotation), priority.

metrics-server — the in-cluster usage pipeline (kubecttl top, HPA Resource metrics).

ttlSecondsAfterFinished — automatic cleanup age for finished Jobs.

JOB_COMPLETION_INDEX — env var in Indexed Jobs for sharded work queues.

[↑ back to top](#)

**WORKLOADS — COMMANDS**

```
$ kubectl get ds,job,cronjob,hpa
$ kubectl create job test --from=cronjob/x
$ kubectl set image ds/agent img=v2
$ kubectl rollout status ds/agent
$ kubectl autoscale deploy web --min=2 --max=10 --cpu-percent=60
$ kubectl get hpa -w
$ kubectl top pods # needs metrics-server
```

STS CANARY PATTERN

```
updateStrategy:
  rollingUpdate: { partition: N-1 }
# set image → top pod only
# verify → lower partition
2 → 1 → 0
# roll = highest ordinal first
# (replicas before primary)
```

SCALER CHOOSER

Symptom	Scaler
load spikes, pods pegged	HPA (replicas)
pods sized wrong, steady	VPA (requests)
nothing fits, Pending	Cluster Autoscaler (nodes)
HPA + VPA same metric	✗ feedback war — split signals

FIELD QUICK-REF

```
Job: completions · parallelism ·
backoffLimit · activeDeadlineSeconds ·
ttlSecondsAfterFinished · restartPolicy Never/OnFailure
CronJob: schedule · concurrencyPolicy
(Forbid for backups!) · startingDeadlineSeconds
HPA math: ceil(replicas × used/target)
±10% tolerance · clamped min/max
```

FACTS WORTH A JOB OFFER

DaemonSet: node list = replica count; auto-tolerates infra taints but NOT control-plane; drain skips its Pods

Jobs count successes and bound failures — restartPolicy must be Never/OnFailure; CronJob's real decision is Allow vs Forbid vs Replace

StatefulSet rollouts: highest-ordinal-first; partition = graduated canary, frozen while partition = replicas

HPA needs requests + metrics-server; utilization = usage ÷ request; fast up, 5-min-stabilized down

VPA right-sizes by eviction; HPA+VPA on the same metric = war — use custom metrics + resources instead

CA buys nodes for Pending Pods and sells idle ones — PDBs, local storage, and priority are its fences

[↑ back to top](#)



Books 01–05 built, dressed, connected, placed, and persisted your workloads — Book 06 **completed the zoo and taught it to run itself.**

THE FULL WORKLOAD ZOO:

└─ Deployment	stateless replicas · rolling + undo (Book 01)
└─ DaemonSet	agent per node · node list = count
└─ Job / CronJob	run-to-done · schedules · concurrency policies
└─ StatefulSet	identity + storage + ORDERED + partition canaries
└─ Autoscaling	HPA (count) · VPA (size) · CA (nodes) – a stack

✓ You can now...

Ship node agents as DaemonSets — and explain why your log collector and CNI had no choice

Run batch work with real failure semantics, and schedule backups that can't overlap themselves to death

Roll database images with partition canaries — one replica verified before the primary ever moves

Wire an HPA the right way (requests + metrics-server) and predict its replica math

Choose among HPA/VPA/CA per question, and stack them without feedback wars



Where to next — Book 07 · Observability & Operations: your workloads now scale themselves — but *how do you know they're healthy?* Probes (the hole Book 01 flagged), Metrics Server & `kubect1 top`, Prometheus + Grafana, logging architecture, and cluster-scope debugging. Your own Loki/Prometheus/Grafana stack maps straight onto it — now you'll see the whole discipline from the Kubernetes side.

— End of Book 06 ·  —